CSESoc Education

COMP2521

Problem Solving Workshop

Linked Lists • Binary Search Trees • Graphs
Hash Tables • Stacks & Queues

●○○ Easy
●●○ Medium
●●● Challenging

https://github.com/itsJae/COMP2521-problem-solving-workshop

Contents

Getting Started **2**

1 Linked List **3**

1.1 ●○○ Is Sorted 3

1.2 ●●○ Copy Alternating Range 4

1.3 ●●● Delete Mid 5

2 Binary Search Tree **6**

2.1 ●○○ Count Leaves 6

2.2 ●●○ Tree Diameter 7

2.3 ●●● Structural Difference 8

3 Graph Traversal **9**

3.1 ●○○ Reach Darling Square 9

3.2 ●●○ Find Happy Component 10

3.3 ●●● Celebrity 11

4 Hash Table **12**

4.1 ●○○ Molly Tea 12

4.2 ●●○ Isomorphic Strings 13

4.3 ●●● Longest Substring Without Repeating Characters 14

5 Stack / Queue **15**

5.1 ●○○ Crunch Pringles 15

5.2 ●●○ Reverse Queue 16

5.3 ●●● Daily Temperatures 17

6 Bonus **18**

6.1 ●●● Project Selection 18

Getting Started

Setup

Install VS Code, Git and a C compiler (see the `README` for each OS). Then open the Command Palette in VS Code (`Cmd/Ctrl + Shift + P`), run **Git: Clone** and paste:

```
https://github.com/itsJae/COMP2521-problem-solving-workshop.git
```

Solving a problem

Each problem is a single C function in its topic folder, for example `linked_list/is_sorted.c`. Fill in the `TODO`. You may write helper functions, and you may use any ADT in the `adt/` folder.

Testing

Open a terminal in VS Code, move into the topic folder and run `autotest`:

```
cd linked_list
./autotest is_sorted # test one problem
./autotest # test every problem in this folder
```

A test fails if your answer is wrong, if your program crashes, or if it takes longer than 5 seconds. Some tests are large, so a solution that is slower than the required time complexity may time out.

To debug with your own `printf`s, build with `make` and run the problem directly. This only shows your own output:

```
make
./is_sorted
```

Notation

In the samples, a linked list is written as `1 -> 2 -> 3 -> X`, where `X` is `NULL`. A BST is given by the order in which values are inserted into an empty tree, together with a drawing of the result.

Back to contents

Is Sorted

File	`linked_list/is_sorted.c`
Test	`./autotest is_sorted`
Required time complexity	$O(n)$

Background

The COMP2521 tutors keep every student's lab mark in a linked list. Their marking script only works if the marks are already in order, so before running it they want a quick check.

Task

Return `true` if the values in the list are in non-decreasing order (every value is less than or equal to the next one), otherwise return `false`.

```
struct node {
    int value;
    struct node *next;
};

bool isSorted(struct node *list);
```

Assumptions

- The list may be empty (`NULL`). An empty list and a list with one node are sorted.
- You must not use `for` or `while` loops. Use recursion instead.
- Equal neighbouring values are allowed.
- The list must not be modified.

Samples

Sample Input 1
`1 -> 2 -> 3 -> 4 -> X`

Sample Output 1
`true`

Sample Input 2
`3 -> 1 -> 2 -> 4 -> X`

Sample Output 2
`false`

Explanation: `3` is greater than the next value `1`.

Back to contents

Copy Alternating Range

●●○

File	`linked_list/copy_alternating_range.c`
Test	`./autotest copy_alternating_range`
Required time complexity	$O(n)$

Background

Your playlist is a linked list where each node holds a song's energy level. Tonight you only want songs with energy between `lo` and `hi`, and to keep things fresh you skip every second song that qualifies.

Task

Considering only the values that lie in the inclusive range $[lo, hi]$, return a **new** list containing the 1st, 3rd, 5th, ... of them, in their original order.

```c
struct node {
    int value;
    struct node *next;
};

struct node *copyAlternatingRange(struct node *list, int lo, int hi);
```

Assumptions

- $lo \leq hi$.
- The list may be empty. If no value lies in the range, return `NULL`.
- The new list must be built from newly allocated nodes. The original list must not be modified.

Samples

Sample Input 1

```
8 -> 3 -> 6 -> 2 -> 7 -> 5 -> 9 -> 4 -> X
lo = 3, hi = 8
```

Sample Output 1

```
8 -> 6 -> 5 -> X
```

Explanation: The values in $[3, 8]$ are `8, 3, 6, 7, 5, 4`. The 1st, 3rd and 5th of them are `8, 6, 5`.

Sample Input 2

```
10 -> 4 -> 20 -> 6 -> X
lo = 4, hi = 6
```

Sample Output 2

```
4 -> X
```

Explanation: Only `4` and `6` are in range, and `6` is the 2nd of them.

Back to contents

Delete Mid



File	`linked_list/delete_mid.c`
Test	`./autotest delete_mid`
Required time complexity	$O(n)$

Background

The waiting line for the CSESoc pubcrawl at Sanctuary Hotel is getting too long. Whoever is standing in the exact middle has been chosen to leave and grab more snacks.

Task

Delete the middle node of the list and return the head of the updated list. For a list of length n, the middle node is the one at index $\lfloor n/2 \rfloor$, counting from 0.

```
struct node {
    int value;
    struct node *next;
};

struct node *deleteMid(struct node *list);
```

Assumptions

- The list is not empty ($n \geq 1$). If it has one node, the result is `NULL`.
- The deleted node must be freed.

Samples

Sample Input 1
`1 -> 3 -> 5 -> 7 -> 9 -> X`

Sample Output 1
`1 -> 3 -> 7 -> 9 -> X`

Explanation: $n = 5$, so the node at index 2 (`5`) is deleted.

Sample Input 2
`1 -> 2 -> 3 -> 4 -> X`

Sample Output 2
`1 -> 2 -> 4 -> X`

Explanation: $n = 4$, so the node at index 2 (`3`) is deleted.

Back to contents

Count Leaves

●○○

File	`bst/count_leaves.c`
Test	`./autotest count_leaves`
Required time complexity	$O(n)$

Background

Autumn has arrived on the library lawn, and the BST growing there is about to drop its leaves. Before the gardeners show up, they want to know how many leaves to expect.

Task

Return the number of leaf nodes (nodes with no children) in the given BST.

```c
struct node {
    int value;
    struct node *left;
    struct node *right;
};

int countLeaves(struct node *tree);
```

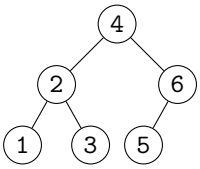
Assumptions

- The tree may be empty, in which case there are 0 leaves.
- A tree with a single node has 1 leaf.

Samples

Sample Input 1

```
insert 4 2 6 1 3 5
```



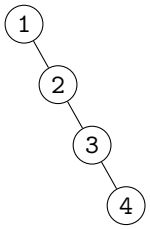
Sample Output 1

```
3
```

Explanation: The leaves are `1`, `3` and `5`.

Sample Input 2

```
insert 1 2 3 4
```



Sample Output 2

```
1
```

Explanation: The tree is a single path, so `4` is the only leaf.

Back to contents

Tree Diameter

●●○

File	`bst/tree_diameter.c`
Test	`./autotest tree_diameter`
Required time complexity	$O(n)$

Background

A squirrel lives in the library BST. It wants to plan the longest possible trip between two nodes without visiting any node twice.

Task

Return the diameter of the tree: the number of **edges** on the longest path between any two nodes.

```c
struct node {
    int value;
    struct node *left;
    struct node *right;
};

int treeDiameter(struct node *tree);
```

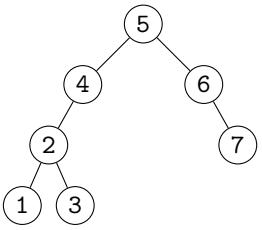
Assumptions

- The tree may be empty. An empty tree and a tree with one node both have diameter 0.
- The longest path does not have to pass through the root.

Samples

Sample Input 1

```
insert 5 4 2 1 3 6 7
```



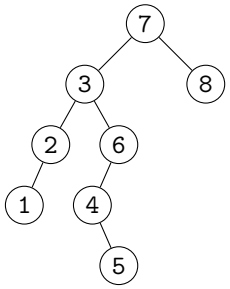
Sample Output 1

```
5
```

Explanation: The path `1-2-4-5-6-7` has 5 edges and goes through the root.

Sample Input 2

```
insert 7 3 2 1 6 4 5 8
```



Sample Output 2

```
5
```

Explanation: The path `1-2-3-6-4-5` has 5 edges and does not go through the root.

Back to contents

Structural Difference

●●●

File	`bst/structural_difference.c`
Test	`./autotest structural_difference`
Required time complexity	$O(n+m)$, where n and m are the numbers of nodes in the two trees

Background

Two students submitted BSTs for the same lab exercise. Their tutor plays spot the difference to measure how far apart the two trees are.

Task

For every position that occurs in at least one of the two trees:

- add 0 if both trees have a node there and the values are equal,
- add 1 if both trees have a node there but the values are different,
- add 2 if exactly one tree has a node there.

Return the total.

```
int structuralDifference(struct node *t1, struct node *t2);
```

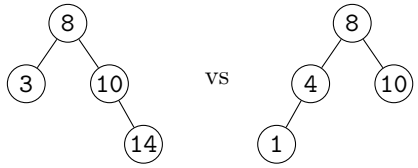
Assumptions

- A position is the path of left/right moves from the root, not a value.
- Either tree, or both, may be empty.
- If one tree has a whole subtree where the other tree has no node, every node in that subtree adds 2.

Samples

Sample Input 1

```
t1:  insert 8 3 10 14
t2:  insert 8 4 10 1
```



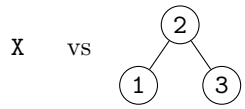
Sample Output 1

```
5
```

Explanation: Root `8 = 8` adds 0, `3` vs `4` adds 1, `1` and `14` each exist in only one tree and add 2.

Sample Input 2

```
t1:  (empty)
t2:  insert 2 1 3
```



Sample Output 2

```
6
```

Explanation: `t1` is empty, so all 3 nodes of `t2` add 2 each.

Back to contents

Reach Darling Square

●○○

File	`graph/reach_darling_square.c`
Test	`./autotest reach_darling_square`
Required time complexity	$O(V + E)$

Background

You and your friends are meeting for dinner at Darling Square. The city is a graph: each vertex is a location and each edge is a direct route between two locations. Before you leave, you want to know whether you can get there at all.

Task

Return `true` if `dest` (the darling square) can be reached from `src`, otherwise return `false`.

```
bool reachDarlingSquare(Graph g, int src, int dest);

// Useful ADT functions
int GraphNumVertices(Graph g);
int GraphDegree(Graph g, int v);
int GraphNeighbour(Graph g, int v, int i); // 0 <= i < GraphDegree(g, v)

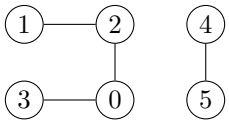
Queue QueueNew(void);
void QueueEnqueue(Queue q, void *item);
void *QueueDequeue(Queue q);
bool QueueIsEmpty(Queue q);
void QueueFree(Queue q, void (*freeItem)(void *item));
```

Assumptions

- The graph is undirected and may be disconnected. Vertices are numbered 0 to $V - 1$.
- $0 \leq src, dest < V$. If $src = dest$, the answer is `true`.

Samples

Both samples use this graph:



Sample Input 1

```
V = 6
edges:  1-2 2-0 0-3 4-5
src = 1, dest = 3
```

Sample Output 1

```
true
```

Explanation: `1 - 2 - 0 - 3` is a path from 1 to 3.

Sample Input 2

```
V = 6
edges:  1-2 2-0 0-3 4-5
src = 1, dest = 5
```

Sample Output 2

```
false
```

Explanation: Vertex 5 is in a different component from vertex 1.

Back to contents

Find Happy Component

●●○

File	`graph/find_happy_component.c`
Test	`./autotest find_happy_component`
Required time complexity	$O(V + E)$

Background

At the end-of-term party, people cluster into friend groups: two people are in the same group if they are connected by a chain of friendships. A group is only happy if it is big enough.

Task

A connected component is *happy* if and only if it has at least k vertices. Return how many components of the graph are happy.

```
int findHappyComponent(Graph g, int k);

// Useful ADT functions
int GraphNumVertices(Graph g);
int GraphDegree(Graph g, int v);
int GraphNeighbour(Graph g, int v, int i); // 0 <= i < GraphDegree(g, v)

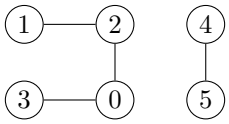
Stack StackNew(void);
void StackPush(Stack s, void *item);
void *StackPop(Stack s);
bool StackIsEmpty(Stack s);
void StackFree(Stack s, void (*freeItem)(void *item));
```

Assumptions

- The graph is undirected. A vertex with no edges is a component of size 1.
- $k \geq 1$.

Samples

Both samples use this graph:



Sample Input 1

```
V = 6
edges:  1-2 2-0 0-3 4-5
k = 3
```

Sample Output 1

```
1
```

Explanation: The components have sizes 4 and 2. Only the first has at least 3 vertices.

Sample Input 2

```
V = 6
edges:  1-2 2-0 0-3 4-5
k = 2
```

Sample Output 2

```
2
```

Explanation: Both components have at least 2 vertices.

Back to contents

Celebrity

●●●

File	`graph/celebrity.c`
Test	`./autotest celebrity`
Required time complexity	$O(n)$

Background

Rumour has it that a celebrity has sneaked into the party. Everyone knows the celebrity, but the celebrity does not know anyone. You may only ask questions of the form "does person i know person j?". Based on The Celebrity Problem.

Task

There are n people at the party, and `mat[i][j] == 1` if person i knows person j. A *celebrity* is known by everyone else but knows no one else. Return the index of the celebrity, or -1 if there is none.

```
int celebrity(int n, int mat[n][n]);
```

Assumptions

- $n \geq 1$. A party with one person has that person as the celebrity.
- `mat[i][i]` may be 0 or 1 and should be ignored.
- Reading the whole matrix takes $O(n^2)$, so you must skip most of it. Hint: each question rules out one person.

Samples

Sample Input 1

```
n = 3
mat:
0 1 0
0 0 0
0 1 0
```

Sample Output 1

```
1
```

Explanation: Everyone knows person 1, and person 1 knows no one.

Sample Input 2

```
n = 2
mat:
0 1
1 0
```

Sample Output 2

```
-1
```

Explanation: Persons 0 and 1 know each other, so neither is a celebrity.

Back to contents

Molly Tea

●○○

File	`hashmap/molly_tea.c`
Test	`./autotest molly_tea`
Required time complexity	$O(n)$ on average

Background

James is a poor uni student on a first date with Julia. He takes her to Molly Tea, opens his wallet, and finds exactly k dollars. He wants to buy two drinks that add up to exactly k dollars so he can spend every last dollar to impress her. Julia gets impatient if James takes too long.

Task

Given the drink prices, find two different drinks whose prices add up to exactly k. If they exist, store their indices in `*i` and `*j` and return `true`. Otherwise return `false`.

```
bool mollyTea(int prices[], int n, int k, int *i, int *j);

// Useful ADT functions (see Common.h for HashInt and EqualsInt)
HashTable HashTableNew(HashFn hash, EqualsFn equals);
void HashTablePut(HashTable t, void *key, void *value);
void *HashTableGet(HashTable t, void *key); // NULL if absent
void HashTableFree(HashTable t, void (*freeKey)(void *key),
                   void (*freeValue)(void *value));
```

Assumptions

- $n \geq 0$. Two different drinks may have the same price, but one drink cannot be bought twice.
- If several pairs work, any one of them is accepted.

Samples

Sample Input 1

```
prices = [2, 7, 11, 15]
k = 9
```

Sample Output 1

```
true
i = 0, j = 1
```

Explanation: $2 + 7 = 9$.

Sample Input 2

```
prices = [3, 4]
k = 6
```

Sample Output 2

```
false
```

Explanation: $3 + 3 = 6$, but there is only one drink that costs 3.

Back to contents

Isomorphic Strings

●●○

File	`hashmap/isomorphic_strings.c`
Test	`./autotest isomorphic_strings`
Required time complexity	$O(n)$ on average

Background

Your friend sends you a secret message `t` and claims it is the word `s` encrypted with a substitution cipher, where each letter is always swapped for the same letter. Before you start decoding, check whether that is even possible. Based on LeetCode 205.

Task

Two strings s and t are isomorphic if the characters in s can be replaced to get t, where every occurrence of a character is replaced with the same character, and no two different characters map to the same character. Return `true` if s and t are isomorphic.

```
bool isIsomorphic(char *s, char *t);
```

Assumptions

- s and t have the same length n, which may be 0. Two empty strings are isomorphic.
- The strings contain ASCII characters. A character may map to itself.

Samples

Sample Input 1	Sample Output 1
`s = "egg"` `t = "add"`	`true`

Explanation: `e` maps to `a` and `g` maps to `d`.

Sample Input 2	Sample Output 2
`s = "badc"` `t = "baba"`	`false`

Explanation: Both `b` and `d` would have to map to `b`.

Longest Substring Without Repeating Characters ●●●

File	`hashmap/longest_substring.c`
Test	`./autotest longest_substring`
Required time complexity	$O(n)$ on average

Background

Your music history is a string where each character is a song. You want to brag about your longest listening streak without a single repeat. Based on LeetCode 3.

Task

Return the length of the longest substring of s that contains no repeated characters.

```
int lengthOfLongestSubstring(char *s);
```

Assumptions

- s may be empty, in which case the answer is 0.
- s contains ASCII characters, including spaces.
- A substring is contiguous: `"pwke"` is not a substring of `"pwwkew"`.

Samples

Sample Input 1	Sample Output 1
`s = "abcabcbb"`	`3`

Explanation: The longest such substring is `"abc"`.

Sample Input 2	Sample Output 2
`s = "pwwkew"`	`3`

Explanation: The longest such substring is `"wke"`.

Back to contents

Crunch Pringles

●○○

File	`stack_queue/crunch_pringles.c`
Test	`./autotest crunch_pringles`
Required time complexity	$O(n^2)$

Background

A Pringles tube only opens at the top, so it behaves like a stack. Each chip has an integer crunch score, and you want to save the crunchiest chip for last.

Task

Sort the tube in place so that scores never decrease from bottom to top (the crunchiest chip ends up on top). You may use `bowl`, an empty stack, as temporary storage.

```
void sortTube(Stack tube, Stack bowl);

// Useful ADT functions (every item is an int *)
void StackPush(Stack s, void *item);
void *StackPop(Stack s);
void *StackPeek(Stack s);
bool StackIsEmpty(Stack s);
```

Assumptions

- The tube may be empty. Scores may be negative or repeated.
- Apart from the two stacks, use only a constant number of variables (no arrays or other ADTs).
- Move the existing items. Do not allocate new ones.

Samples

Sample Input 1

```
tube (bottom -> top):
3 1 4 2
```

Sample Output 1

```
tube (bottom -> top):
1 2 3 4
```

Sample Input 2

```
tube (bottom -> top):
5 5 2
```

Sample Output 2

```
tube (bottom -> top):
2 5 5
```

Back to contents

Reverse Queue

●●○

File	`stack_queue/reverse_queue.c`
Test	`./autotest reverse_queue`
Required time complexity	$O(n)$

Background

It is happy hour at the food court, and the manager announces that whoever joined the queue last will be served first.

Task

Reverse the order of the items in the queue in place, so the item at the back ends up at the front.

```
void reverseQueue(Queue q);

// Useful ADT functions (every item is an int *)
void QueueEnqueue(Queue q, void *item);
void *QueueDequeue(Queue q);
bool QueueIsEmpty(Queue q);

Stack StackNew(void);
void StackPush(Stack s, void *item);
void *StackPop(Stack s);
bool StackIsEmpty(Stack s);
void StackFree(Stack s, void (*freeItem)(void *item));
```

Assumptions

- The queue may be empty. Values may be repeated.
- Move the existing items. Do not allocate new ones.

Samples

Sample Input 1

```
q (front -> back):
1 2 3 4 5
```

Sample Output 1

```
q (front -> back):
5 4 3 2 1
```

Sample Input 2

```
q (front -> back):
3 3 1
```

Sample Output 2

```
q (front -> back):
1 3 3
```

Back to contents

Daily Temperatures

●●●

File	`stack_queue/daily_temperatures.c`
Test	`./autotest daily_temperatures`
Required time complexity	$O(n)$

Background

Sydney weather is unpredictable, and you are planning your beach days. For every day, you want to know how long you would have to wait for a warmer one. Based on LeetCode 739.

Task

`temperatures[i]` is the temperature on day i. For each day, set `answer[i]` to the number of days you have to wait after day i to get a warmer temperature, or 0 if there is no such day.

```
void dailyTemperatures(int temperatures[], int n, int answer[]);
```

Assumptions

- $0 \le n \le 200\,000$, and `answer` has room for n values.
- An equal temperature is not warmer.
- An $O(n^2)$ solution is too slow for the largest test. Hint: a stack of days that are still waiting for a warmer day.

Samples

Sample Input 1

```
temperatures =
[73, 74, 75, 71, 69, 72, 76, 73]
```

Sample Output 1

```
answer =
[1, 1, 4, 2, 1, 1, 0, 0]
```

Explanation: Day 2 (`75`) waits until day 6 (`76`), so `answer[2] = 4`.

Sample Input 2

```
temperatures =
[30, 40, 50, 60]
```

Sample Output 2

```
answer =
[1, 1, 1, 0]
```

Back to contents

Project Selection



File	`bonus/project_selection.c`
Test	`./autotest project_selection`
Required time complexity	$O(n \log k)$

Background

You are planning your degree. Each project needs some knowledge before you can start it, and teaches you more once you finish. You only have time for k projects, so choose wisely.

Task

You are given two arrays `R` and `W`, and two other inputs s and k:

- `R[i]` is the knowledge required to select project i,
- `W[i]` is the knowledge gained after selecting project i,
- s is your starting knowledge, and k is the number of projects to select.

Your knowledge grows by `W[i]` after each project you select, and you cannot select the same project twice.

(a) Which data structure solves the problem in $O(n \log k)$ time?

A. Stack
B. Queue
C. Unordered array
D. AVL tree
E. Heap

(b) Return the maximum knowledge you can have after selecting k projects.

```
int projectSelection(int R[], int W[], int n, int s, int k);
```

Assumptions

- `R` is sorted in ascending order.
- $1 \leq k \leq n$, and it is always possible to select k projects.

Samples

Sample Input 1	Sample Output 1

`R = [1, 2, 2, 3, 4]`
`W = [2, 3, 1, 4, 10]`
`s = 2, k = 3`
	`19`

Explanation: Select projects 1, 4 and 3: $2 \to 5 \to 15 \to 19$.

Sample Input 2	Sample Output 2

`R = [0, 2, 5, 10]`
`W = [5, 1, 10, 2]`
`s = 0, k = 3`
	`17`

Explanation: Select projects 0, 2 and 3: $0 \to 5 \to 15 \to 17$.